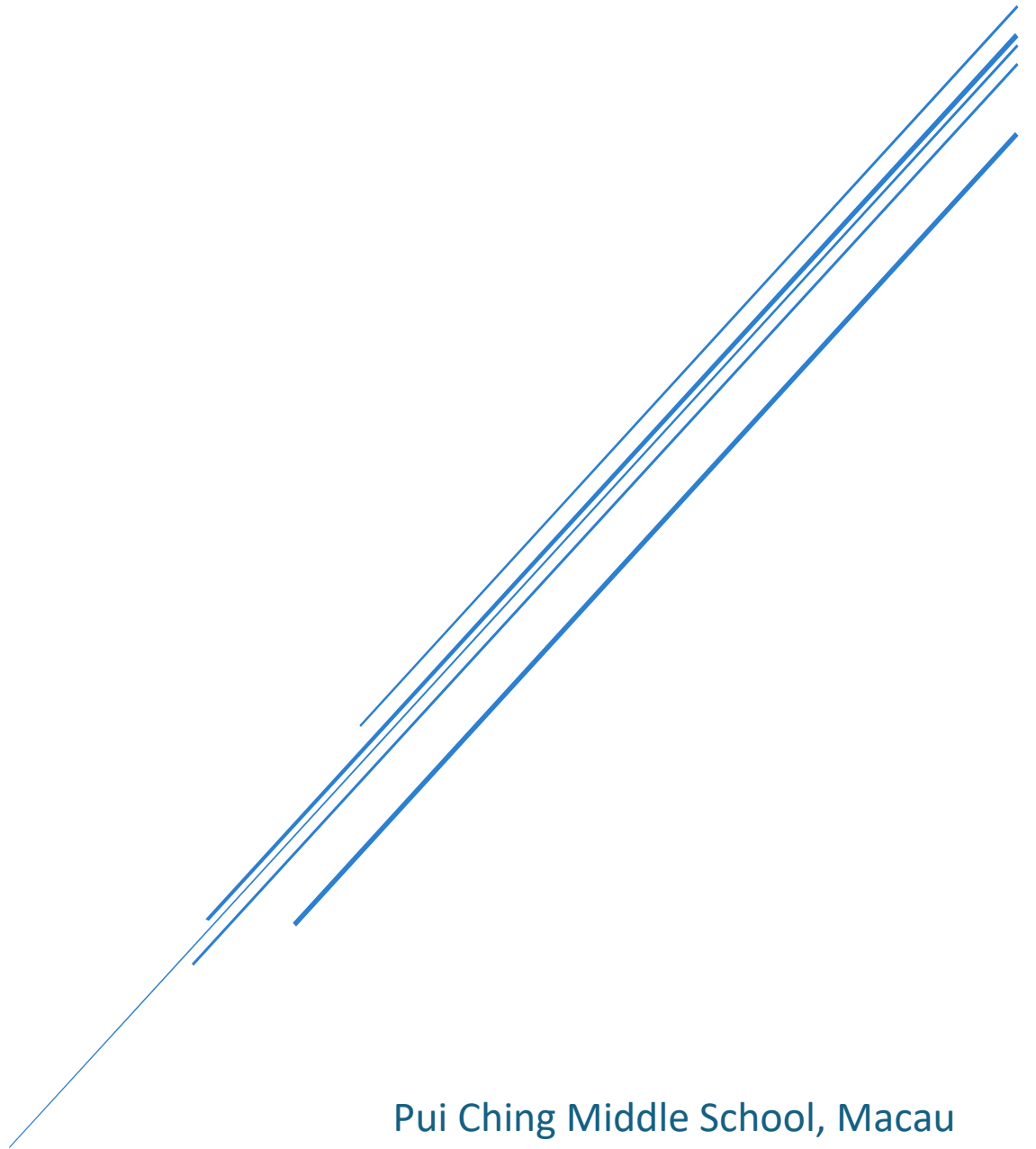


PCMS_ET ENGINEERING JOURNAL

ROBOCUP 2026 INCHEON



Pui Ching Middle School, Macau

I. Executive Summary

Our team has been committed to preparing for the RoboCupJunior Rescue Maze competition. We designed and developed two generations of robots with the goal of carrying out rescue missions efficiently in complex maze environments, locating as many victims as possible and accurately deploying rescue kits.

The first-generation robot was mainly used for functional verification and structural testing. It used a 3D-printed chassis, a JoinMax Digital X4 control board, miniature metal motors, ultrasonic sensors, color sensors, and other basic modules. It initially achieved maze movement and obstacle detection. However, after multiple rounds of real-world testing, we found that the first version had several problems, including unstable power output, insufficient sensing accuracy, cluttered circuit layout, and a high center of gravity.

Based on the testing feedback from the first version, we carried out a comprehensive structural redesign and developed the second-generation robot. The main optimizations included replacing the power system with GA25-370 miniature DC geared motors to solve the gear disengagement problem; upgrading the sensing module to laser sensors to improve distance-measurement accuracy and stability; replacing the control system with a smaller ESP-32 control board; and independently designing two dedicated PCB boards to simplify wiring and improve system stability. In addition, the wheel structure was improved with an annular groove to prevent the silicone tire layer from detaching, and the electronics box was moved from the upper rear of the vehicle to the center of the underside, effectively lowering the center of gravity, balancing the weight distribution, and improving climbing and driving stability.

In terms of software, we initially developed the program using the RoboEXP graphical programming environment. Later, in response to the hardware upgrades in the second-generation robot and the increased need for program

flexibility, easier debugging, and more complex logic processing, we switched to Thonny. The program covers color recognition, motor driving, servo control, maze exploration, path recording, victim detection, and other functions, enabling the robot to complete core tasks such as autonomous navigation, obstacle avoidance, coordinate positioning, and rescue-kit deployment.

Through the iterative design and continuous optimization of the two generations of robots, we significantly improved the robot's performance in power output, environmental perception, structural strength, and driving reliability, laying a solid foundation for completing rescue missions stably and efficiently in competition.

Throughout the entire project, we placed strong emphasis on teamwork and innovation. Each member fully contributed their unique skills in mechanical design, programming, and other areas. Together, we created an organized project plan covering key development stages such as mechanical design, hardware assembly, and algorithm optimization. To ensure project success, we repeatedly tested the robot and made timely improvements based on testing feedback and performance evaluation results.

1. Introduction

a. Team



Team Leader Lao Chi leng:

- Hardware assembly
- Circuit assembly and design
- Software programming

As the team leader, I led the overall robot development and optimization process. On the hardware side, I was responsible for the 3D model design and physical assembly of the robot's overall structure, and I led the overall planning and safety testing of the circuit system. On the software side, I

personally wrote the robot's core control and kinematic logic. With spatial reasoning and mechanical construction experience gained from many previous robotics competitions, I was able to plan the most suitable hardware configuration accurately. In addition to my own tasks, I also coordinated technical work across subgroups, helped teammates refine programming logic and debugging, and supervised the development progress and pre-competition maintenance to ensure that the robot could demonstrate its best performance on the RoboCup field.



Lui Iek Tong:

- Circuit assembly and design
- Documentation and operation log recording

In the team, I was mainly responsible for the detailed construction of the circuit system, wiring-layout optimization, and hardware soldering, ensuring stable robot power supply and signal transmission. I also took charge of document writing and engineering-log recording, documenting the team's daily research and development, competition preparation, and competition-site activities. I was responsible for taking high-quality photos and videos, producing daily team short videos and important event records, and operating the team's official account to showcase our work. In addition, with strong English communication skills, I was

responsible for reviewing the latest RoboCup international rules and handling international communication and technical Q&A during the competition.



Lei Un Hou:

- Software programming
- Documentation and operation logs

In the team, I specialized in robot software coding and system function development. I began learning Python in primary school and studied C programming in depth during secondary school, developing a strong passion for programming and algorithm implementation. In this project, I focused on turning code logic into highly executable functional designs and was responsible for writing the low-level drivers and code for each functional module. At the same time, I recorded the team's operation logs, carefully tracking and archiving each code iteration and debugging result so that every development milestone, technical detail, and troubleshooting process could be properly documented as a complete technical asset for the team.



Wong Chi Kio:

- Image recognition
- Software programming

In the team, I was mainly responsible for developing the robot's vision system and debugging both software and hardware. Drawing on my Python experience since primary school and my deeper study of C during secondary school, I applied my passion for programming to the highly challenging field of image recognition. My main responsibility was to develop target detection so that the robot could accurately recognize key victims on the competition field. During debugging, I acted as a bridge between the software group and the electrical-control group, carrying out comprehensive system integration. By continuously optimizing algorithm feedback speed, I worked to improve the robot's response agility and decision-making accuracy in dynamic competition environments.

This is our talented and charismatic advisor, Lam Tak Kam.



II. Project Planning

1. Overall Project Plan

- August 20, 2025

The main goal of our team was to design and develop a robot capable of performing rescue missions efficiently in complex maze environments, locating as many victims as possible and accurately deploying rescue kits. We also aimed to achieve excellent results in competition and demonstrate our capabilities and innovative spirit in robotics technology and teamwork.

During development, we had to fully consider the constraints set by the competition and plan the entire project accordingly.

First were the physical constraints. The robot height must not exceed 25 cm to ensure that it can move smoothly between maze walls and obstacles without collisions or blockages. At the same time, the robot's sensors and equipment must not be able to “see” above the maze walls, in order to maintain the fairness and challenge of the competition. This required us to be especially careful in structural design and sensor placement so that the robot complied with the rules.

Second were the time constraints. Before the competition, we had to complete

the robot's design, fabrication, programming, and testing within a limited period. During the competition, each round lasts 8 minutes, including calibration and scoring runs. This requires the robot to adapt quickly to the field environment and complete tasks efficiently. Therefore, we established a clear development schedule and testing plan to ensure that each stage progressed in an organized way.

Finally, there were competition rules and restrictions. The robot must operate fully autonomously; remote control, manual control, and external information transmission are prohibited. Before the competition, we must submit technical description documents, an engineering journal, posters, and other materials, and ensure that the robot passes pre-competition inspection. During the competition, we must also follow field rules, such as avoiding damage to the field, correctly identifying victims, and accurately deploying rescue kits. These rules directly influenced our hardware and software design and task-logic planning.

Considering all these constraints, we kept compliance and stability as the core principles throughout development. Every design and optimization decision, from structural improvements to software upgrades, from circuit integration to algorithm adjustment, was made with the competition constraints in mind to ensure that the robot could perform reliably during competition.

2. Key Milestones

- September 05, 2025

Dimension measurement and structural planning: We measured the dimensions of all IoT components and began designing the mechanical structure based on the data. By accurately recording the length, width, height, and mounting locations of each module, we planned the chassis layout in Onshape to ensure that components were securely fixed and that the center of gravity was reasonable, preparing for later hardware assembly.

The robot hardware design included the JoinMax Digital X4 RCU for control,

miniature metal motors to drive the wheels, and ultrasonic sensors for distance measurement. A color sensor identified the color of the maze floor, while a pair of K210 camera lenses detected wall patterns for block placement. A gyroscope ensured directional accuracy, a metal digital servo (MG90) enabled precise steering, and the transformer used electromagnetic induction to convert high-voltage alternating current into safe voltages required by each component.



JoinMax Digital X4 RCU: This control board is used to manage various devices and applications, and it is compatible with many sensors, actuators, and other control modules.



Miniature metal motor: We used it to drive wheel rotation. Its small size and high torque help the vehicle climb slopes and overcome obstacles effectively.



Ultrasonic sensor: Using sonar principles, it provides excellent non-contact distance measurement and helps us accurately determine the distance between the robot and the maze walls.



Color sensor: We installed the color sensor on the underside of the robot to detect and identify the color of the maze floor, helping the vehicle decide its next action.



K210 camera lens: It can recognize images within milliseconds. We installed one lens on each side of the robot to recognize patterns on the maze walls so that blocks can be placed correctly.



Gyroscope: This is used to measure the rotation angle of the robot vehicle. With gyroscope data, the robot can accurately determine its current direction.



MG90 metal digital servo: This small servo contains a high-quality high-speed motor and metal gear transmission. It enables accurate and stable turning, allowing blocks to be placed in the correct position.



Transformer: The transformer uses electromagnetic induction to convert high-voltage alternating current into safe voltages required by other electronic devices.

● September 18, 2025

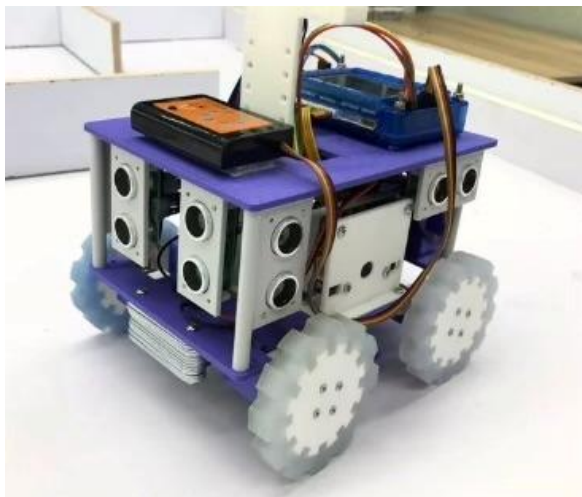
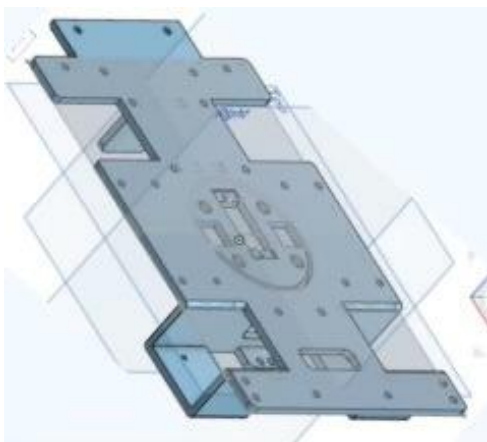
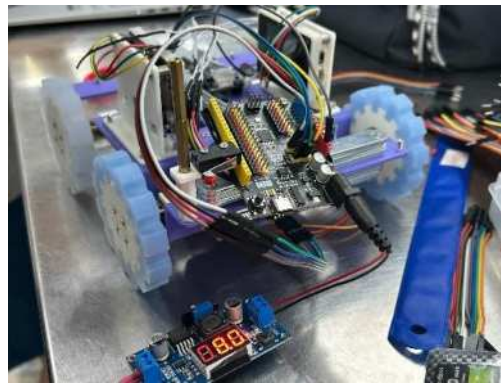
First-generation mechanical structure completed: The robot's appearance and basic structure were formally determined, and the model could begin to be printed. We completed the mechanical structure model, carefully planned the fit and assembly method of each component, and confirmed the overall center-of-gravity distribution and sensor mounting positions, laying a good foundation for later hardware assembly.



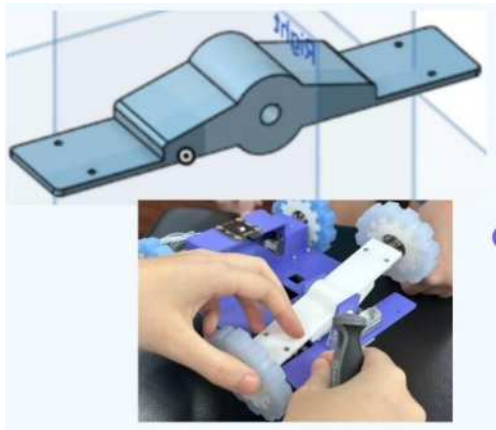
- September 24, 2025

Hardware assembly and sensor installation: After the hardware assembly and sensor installation were completed, the robot's hardware system was ready for initial power-on testing and sensor data collection. At this stage, we needed to check whether all hardware devices operated normally and confirm whether the returned sensor data was accurate, so that subsequent programming and functional testing could proceed smoothly.

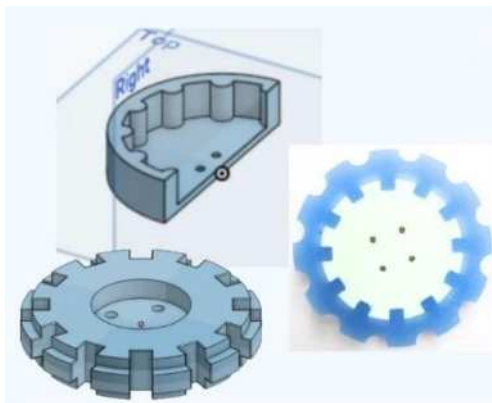
The robot hardware design included the JoinMax Digital X4 RCU for control, miniature metal motors to drive the wheels, and ultrasonic sensors for distance measurement. A color sensor identified the color of the maze floor, while a pair of K210 camera lenses detected wall patterns for block placement. A gyroscope ensured directional accuracy, a metal digital servo (MG90) enabled precise steering, and the transformer used electromagnetic induction to convert high-voltage alternating current into safe voltages required by each component.



- Parallel mechanism: To ensure that the robot vehicle could cross obstacles smoothly, we added a parallel mechanism to the rear wheels. This mechanism keeps all four wheels in contact with the ground at all times and prevents wheels from lifting off when crossing obstacles.

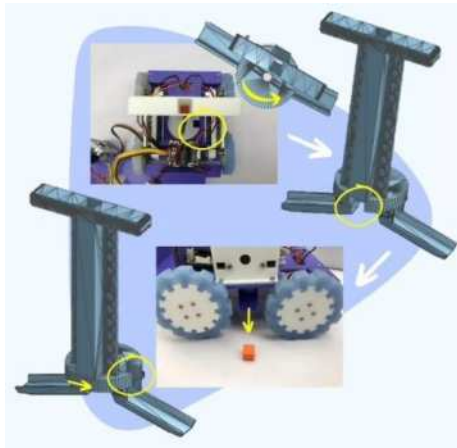


- Wheels: We first created wheel molds using 3D drawing software, then manufactured the robot vehicle wheels using the chemical properties of resin and hardener. This strengthened wheel friction, and the gear-like



wheel design increased the contact area between the wheels and ground obstacles.

- Block placement and rail design: Since blocks must be deployed from both sides, we designed a turntable to control the direction in which the blocks drop. We also installed a rail under the robot to ensure that the blocks fall accurately into the correct position.



- October 14, 2025

Navigation and control algorithm development: Developing the navigation and control algorithms enabled the robot to navigate autonomously and perform tasks in the maze. We designed the main program logic and framework, including core modules for maze exploration, obstacle judgement, and direction decisions, ensuring that the algorithm could accurately identify paths, avoid obstacles, and find targets.

- October 25, 2025

Program completed: We integrated all functional modules, including color sensing, distance detection, motor control, servo deployment, and maze exploration. We then began formal testing by placing the robot in a simulated maze and verifying each module one by one to ensure that the program operated stably and met competition requirements.

We used RoboEXP as the programming software for the first-generation robot. RoboEXP is a graphical programming environment designed for youth education. Based on engineering C and modular programming, it allows users to create flowcharts by dragging and dropping icons. Its intuitive operation

helped us quickly implement the robot's basic control logic.



a. General Software Architecture

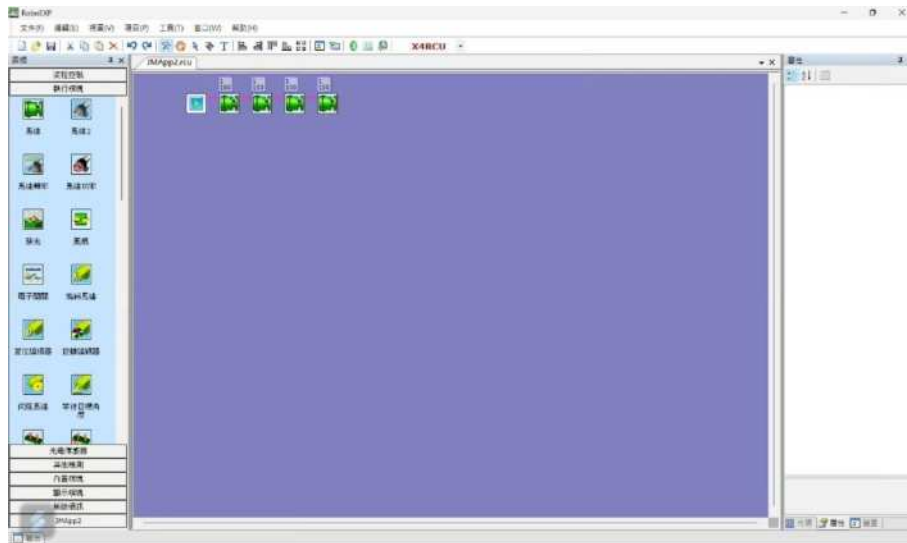
When using RoboEXP to develop the first-generation robot program, we built the main robot flowchart through the user interface by dragging and dropping icons. Steps such as startup, sensor reading, motor control, and decision making were placed into the flowchart as modules and connected in execution order.

We called the required predefined modules from the module library, such as the motor-drive module, ultrasonic-sensor reading module, and color-recognition module. For more complex logic, we used conditional judgement and loop modules to create branching and repeated execution structures.

After completing the flowchart design, the core logic engine automatically converted the flowchart into C source code. This meant that we did not need to write code manually, allowing us to focus on the correctness of the logic and the completeness of the process.

Through the data management system, we saved the completed project and could reload it at any time for modification or optimization. During testing, we repeatedly adjusted the parameters and logic sequence in the flowchart until the robot's behavior matched expectations.

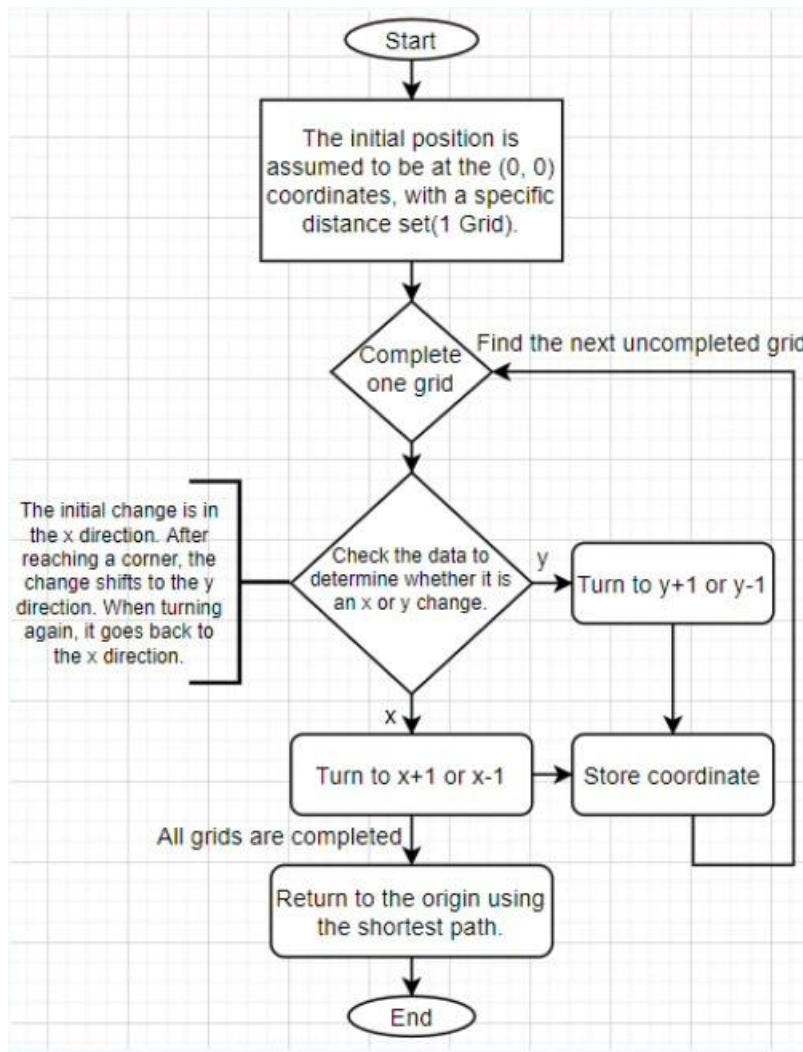
This graphical development approach allowed us to get started quickly and focus on the robot's task logic without spending large amounts of time on



syntax debugging, greatly improving early-stage development efficiency.

b. Innovative Solution

This code implements a simple maze exploration program in C. In the program, 1 represents a wall and 0 represents an open path, and the values are used to record branch points. Each time the robot passes through a grid, it records the path and manages branch points; finally, it uses the analyzed shortest path to return to the starting position.



c. Angle Conversion Issue and Solution

When using a compass for navigation and turning, differences in angle readings may occur. For example, if the robot is currently facing 350 degrees and receives a command to turn right by 50 degrees, the theoretical result is 400 degrees, but the actual compass reading after the turn is 40 degrees. If the robot were directly commanded to turn to 400 degrees, an error would occur because the value is outside the valid angle range.

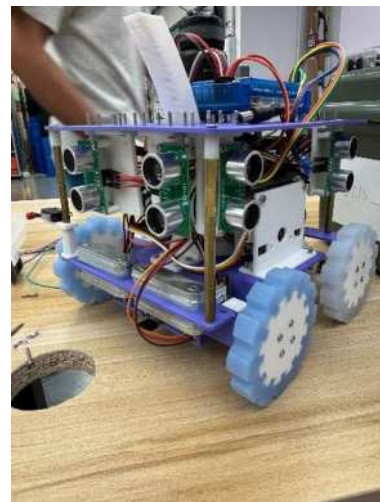
To solve this problem, we used conditional judgement. When this special case occurs, we create a variable to store the correct value calculated by a specific formula. In the example above, adding 350 and 50 and then applying the formula gives the correct value of 40 degrees, allowing the turning angle to be

adjusted accurately. The same principle also applies to left turns, ensuring that the robot can obtain the correct angle value under any turning command.

- November 3, 2025

Overall testing and optimization: After multiple rounds of testing and improvement, the robot reached a good level of performance and stability. At this stage, we needed to conduct comprehensive tests of the robot's speed, accuracy, reliability, and other functions, and optimize the design based on the results.

We considered that the layout of robot components might cause uneven weight distribution and affect climbing performance. Therefore, we placed heavier components, such as batteries, closer to the center and added metal blocks to the underside of the robot to increase weight.



- November 5, 2025

Documentation: During the documentation stage, we began recording the team's work process and results in written form for judges and other teams to understand. The document followed the required format and covered the project background, technical solutions, test results, and other content, presenting the robot's development process and design concept in full.

- December 6, 2025

Improvement planning: Based on the problems found during testing of the first-generation robot, we began planning improvements. The team conducted in-depth discussions and analysis of shortcomings in power, sensing, structure, and other aspects, and initially planned the optimization direction for the second version to prepare for the next design iteration.

Person in charge: Lau Tsz Ying, Lui Yik Tung

- January 19, 2026

Reselection of IoT components: For the second version, we reselected IoT components based on the problems discovered during testing of the first-generation robot. We focused on improving power output and sensing accuracy by replacing key components and adjusting the hardware configuration, effectively enhancing operating stability and environmental perception while laying a solid foundation for later assembly and testing.

During multiple real-world tests of the first-generation robot, we observed that the original miniature metal motors were not stable enough during continuous operation and could not satisfy the requirements for long-term stable work. Gear disengagement, jamming, and other abnormal conditions occurred frequently during use. These problems not only affected normal robot actions but also created structural risks.

Based on this problem, after comparing and screening several options, we

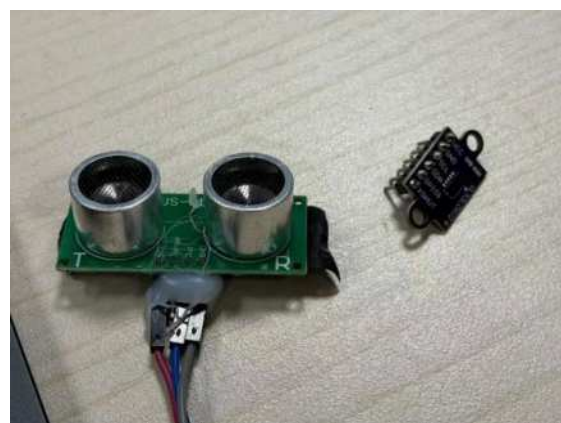
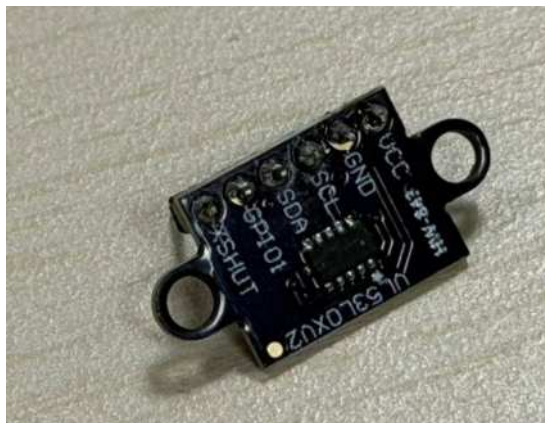


finally decided to replace the original motors with GA25-370 miniature DC geared motors. This motor provides better power output, rotational stability,

and durability. It effectively solves the previous gear disengagement problem and offers significantly better overall performance than the original motor.

We also upgraded the robot's environmental perception and distance-detection modules. Although the ultrasonic sensors used in the first generation could perform basic distance measurement, they still had limitations in accuracy, response speed, and installation flexibility.

To improve environmental recognition and obstacle-avoidance accuracy, we replaced the original ultrasonic sensors with laser sensors. These laser sensors are smaller and easier to mount flexibly on the body. Compared with ultrasonic distance measurement, laser sensors provide higher measurement



accuracy and more stable and reliable values, effectively improving the robot's accuracy during movement, obstacle avoidance, and localization. Overall sensing performance was greatly improved.

- February 08, 2026

PCB design: To optimize control and circuitry, we replaced the oversized original main control board with a more compact ESP-32 control board. At the same time, we independently designed two dedicated PCB boards to

integrate the circuits. This simplified wiring, improved system stability, and made the overall circuit layout of the robot cleaner and more reliable.

During assembly and real-world testing of the first-generation robot, we found that the original main control board was too large. It occupied a large amount of valuable internal space, made the overall structure bulky and loose, hindered later module expansion and wiring organization, and affected the compactness of the robot.

To improve this situation, after comparing several models and testing their performance, we selected a smaller and more streamlined ESP-32 control board to replace the original main control module. While reducing the occupied space, this board provides stronger computing power, stable data-processing efficiency, and good expandability. It can meet the coordinated control requirements of all robot modules and greatly frees up internal space.

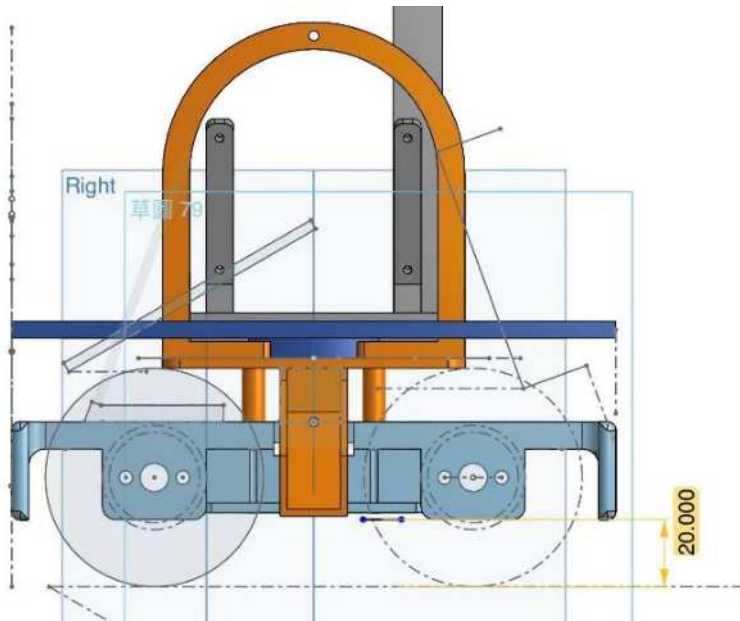
In addition, the general-purpose circuit board used in the first-generation robot had problems such as complicated wiring, low integration, and insufficient stability. During debugging, poor contacts and disorganized wiring occurred easily, increasing later maintenance difficulty. Therefore, we abandoned the standardized circuit-board approach and independently designed and produced two dedicated PCB boards, each responsible for different circuit functions. By integrating and optimizing power supply, signal transmission, and module connections, the internal wiring became more orderly, wiring interference and failure risk were reduced, and the stability and reliability of the robot's overall circuit system were further improved, making the overall structure more compact and practical.



- February 22, 2026

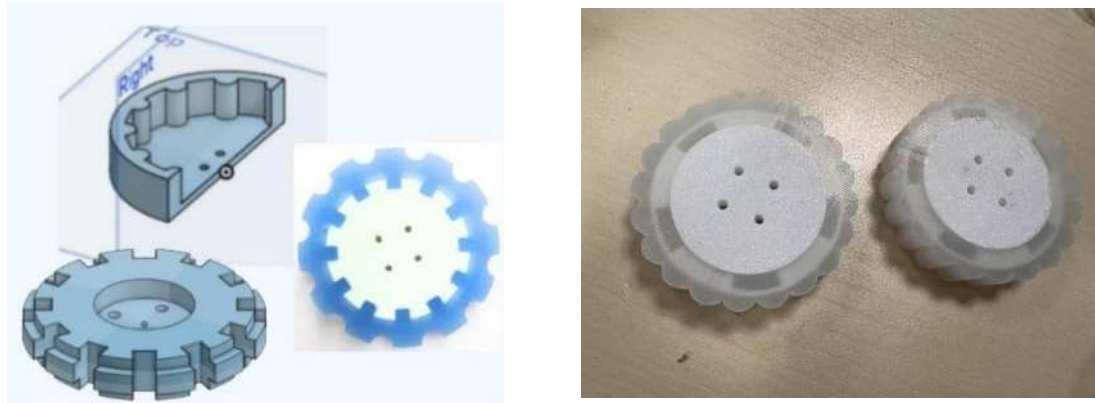
Mechanical structure improvement: During compatibility testing, we found that the GA25-370 miniature DC geared motors were relatively larger and could not fit perfectly into the original body structure. After multiple internal discussions and structural simulations, we finalized the optimization plan: abandon the original parallel-mechanism design and raise the overall body height instead, providing enough installation space for the new motors while maintaining structural coordination and operating stability. The current body

clearance from the ground is 2 cm, allowing the robot to cross speed bumps easily.



Wheel Structure Optimization

During actual testing of the first-generation robot, we found a clear structural defect in the wheels: when the robot turned, braked suddenly, or drove for a long time, the silicone layer attached to the tire could loosen, shift, or even detach. This seriously affected driving stability and steering accuracy, caused



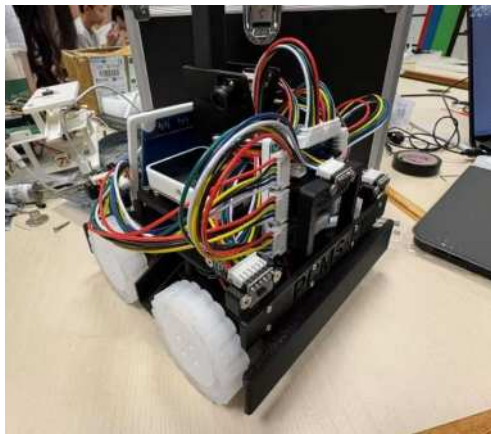
uneven tire wear, greatly shortened service life, and increased maintenance costs.

To completely solve the silicone detachment problem, we carried out targeted structural optimization in the 3D model of the second-generation wheels. We added an annular groove to the wheel body that perfectly matches the silicone tire. With this design, the silicone tire can be fully embedded in the groove, creating a tight engagement between the tire and hub and fundamentally limiting lateral displacement and detachment risk. The groove structure also improves the tightness of the tire-hub connection, strengthens the overall wheel structure, and allows the robot to maintain stable and reliable grip and driving performance during complex scenarios such as turning and climbing. It also extends tire life and reduces the frequency and difficulty of later maintenance.



- March 2, 2026

Second-generation robot fully assembled: The overall assembly of the second-generation robot has been successfully completed. From body structure assembly and hardware component placement to circuit integration, the complete machine is fully assembled with a complete structure and is ready to enter later testing and functional verification.



- March 28, 2026

Program optimization: We changed the programming software from RoboEXP to Thonny and optimized the program comprehensively. Through a more flexible program architecture and more precise control logic, we improved the robot's response speed, sensor-data processing efficiency, and task-execution stability, making the overall operation smoother and more reliable.



The program is mainly used to drive the color sensor and enable the robot to detect environmental color and light intensity. By creating a dedicated class, the program encapsulates low-level operations such as sensor initialization, register reading and writing, and data conversion. It can accurately read raw red, green, and blue values, automatically convert them into standard RGB color data, and calculate the current ambient illumination.

The `get_color_and_lux()` function handles I2C communication configuration, sensor scanning, and data acquisition. It stably outputs RGB values and illumination information, providing support for color recognition, path

```
@property
def color_raw(self):
    """读取原始颜色数据"""
    c = self._read16(0x00 | 0x14) # Clear
    r = self._read16(0x00 | 0x16) # Red
    g = self._read16(0x00 | 0x18) # Green
    b = self._read16(0x00 | 0x1A) # Blue
    return (r, g, b, c)

@property
def color_rgb_bytes(self):
    """返回 RGB 字节值"""
    r, g, b, c = self.color_raw
    if c == 0:
        return (0, 0, 0)
    # 归一化蓝绿通道 8 位
    r = min(255, int((r / c) * 255))
    g = min(255, int((g / c) * 255))
    b = min(255, int((b / c) * 255))
    return (r, g, b)

@property
def color(self):
    """返回 RGB 颜色值"""
    r, g, b = self.color_rgb_bytes
    return (r << 16) | (g << 8) | b

@property
def lux(self):
    """计算亮度"""
    r, g, b, c = self.color_raw
    # 简单的亮度计算
    return c * 0.048 # 调整比例系数以獲得準確的 lux 值
```

```
def get_color_and_lux():
    """
    獲取顏色传感器的 RGB 值和亮度
    返回: (r, g, b, lux) 元组
    """
    try:
        # 使用與OLED相同的I2C引脚 - 创建新的I2C实例
        i2c_color = SoftI2C(scl=Pin(22), sda=Pin(21), freq=100000)

        # 扫描设备
        devices = i2c_color.scan()
        #print("I2C 设备:", [hex(d) for d in devices])

        if 0x29 in devices:
            sensor = TCS34725(i2c_color)

            # 读取颜色和亮度
            color_rgb = sensor.color_rgb_bytes
            lux = sensor.lux

            # 返回 (r, g, b, lux) 元组
            return color_rgb[0], color_rgb[1], color_rgb[2], lux
        else:
            print("未找到 TCS34725 传感器!")
            return (0, 0, 0, 0.0)
    except Exception as e:
        print("颜色传感器读取错误:", e)
        return (0, 0, 0, 0.0)
```

identification, light-source judgement, and other functions. The overall architecture is simple and stable and can be directly integrated into the robot control system.

b. Motors

```
def motor_control(speed_a, speed_b, speed_c, speed_d):
    """
    控制四個馬達的速度和方向
    """
    # 限制速度範圍在-100到100之間
    speed_a = max(-100, min(100, speed_a))
    speed_b = max(-100, min(100, speed_b))
    speed_c = max(-100, min(100, speed_c))
    speed_d = max(-100, min(100, speed_d))

    # 馬達A控制
    if speed_a > 0:
        AIN1_1.on()
        AIN2_1.off()
        PWMA1.duty_u16(int(speed_a * 655.35))
    elif speed_a < 0:
        AIN1_1.off()
        AIN2_1.on()
        PWMA1.duty_u16(int(-speed_a * 655.35))
    else:
        AIN1_1.off()
        AIN2_1.off()
        PWMA1.duty_u16(0)

    # 馬達B控制
    if speed_b > 0:
        BIN1_1.on()
        BIN2_1.off()
        PWMB1.duty_u16(int(speed_b * 655.35))
    elif speed_b < 0:
        BIN1_1.off()
        BIN2_1.on()
        PWMB1.duty_u16(int(-speed_b * 655.35))
    else:
        BIN1_1.off()
        BIN2_1.off()
        PWMB1.duty_u16(0)
```

de
rpt)

```
def stop_all_motors():
    """停止所有馬達"""
    AIN1_1.off()
    AIN2_1.off()
    BIN1_1.off()
    BIN2_1.off()
    PWMA1.duty_u16(0)
    PWMB1.duty_u16(0)

    AIN1_2.off()
    AIN2_2.off()
    BIN1_2.off()
    BIN2_2.off()
    PWMA2.duty_u16(0)
    PWMB2.duty_u16(0)
```

c. Servo

The program is used to drive the servo. By creating a dedicated class, it encapsulates hardware initialization, angle calculation, and PWM signal control. The program first sets the standard 50 Hz operating frequency and defines the corresponding pulse width and conversion parameters based on the servo specifications. The core `set_angle()` function accepts angle values from 0 to 180 degrees, automatically converts the angle to the PWM duty cycle, and precisely controls the servo to rotate to the specified angle for stable positioning. The `deinit()` function releases PWM resources to avoid

power consumption caused by long-term servo locking. The overall program is simple and efficient, and it can be directly used in robot mechanism control, sensor steering, and other scenarios requiring precise angular positioning.

```
#-----SERVO-----
class MG90Servo:
    def __init__(self, pin_number=14):
        self.pwm = PWM(Pin(pin_number))
        self.pwm.freq(50)

        self.min_pulse = 500
        self.max_pulse = 2500

        self.min_duty = int((self.min_pulse / 20000) * 65535)
        self.max_duty = int((self.max_pulse / 20000) * 65535)

    def set_angle(self, angle):
        angle = max(0, min(180, angle))
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)
        duty = int((pulse_width / 20000) * 65535)
        self.pwm.duty_u16(duty)

        #print(f"伺服馬達設定為 {angle} 度")
        return angle

    def deinit(self):
        self.pwm.deinit()

# 創建伺服馬達物件
servo = MG90Servo(16)
#-----SERVO-----
```

2. Main Program

To give the robot a clear concept of orientation and a position reference during movement, we first set the initial coordinates and direction.

Directions are defined numerically: 0 represents north, 1 represents east, 2 represents south, and 3 represents west. This numerical direction rule allows the robot to judge its orientation more accurately when turning, moving forward, and planning paths. Combined with the initial coordinates, it enables stable positioning and movement control.

a. Basic Inspection and Obstacle-Avoidance Judgement Functions

This code implements a series of basic judgement functions for robot maze inspection and obstacle avoidance. All functions use a return mechanism of “return 1 if true, return 0 if false,” making them easy for the main program to call. First, the path-recording functions include `is_visited(x, y)`, which checks whether the input coordinate (x, y) has already been passed by the robot by

comparing it with the global path list, preventing repeated detours. The `is_dropped(x, y)` function checks whether a block has already been deployed at that coordinate to avoid recognizing the same target repeatedly. The remaining three functions detect obstacles in three directions: left, front, and right. By reading the distance values from the corresponding sensors and comparing them with thresholds, they determine whether each direction is free of wall obstruction, providing the basis for later turning and movement decisions.

```
def is_visited(x, y):
    """
    檢查指定座標是否走過
    走過返回 1，否則返回 0
    """
    if [x, y] in visited_path:
        return 1
    return 0

def is_dropped(x, y):
    """
    檢查指定座標是否發現過人質
    發現返回 1，否則返回 0
    """
    if [x, y] in dropped_path:
        return 1
    return 0
```

```
def left_no_wall():
    if d4 > 250:
        return 1
    else:
        return 0

def front_no_wall():
    if d2 > 250:
        return 1
    else:
        return 0

def right_no_wall():
    if d6 > 200:
        return 1
    else:
        return 0
```

b. Gyroscope Closed-Loop Turning and Target Search

This part of the code mainly implements precise closed-loop turning control based on the gyroscope, as well as target searching during the turn. The program reads the gyroscope Z-axis angle in real time to determine the rotation amount. The `turn_left()` and `turn_right()` functions drive the four independent wheel motors to perform the corresponding turning actions according to the target angle. The program also includes a complete angle-boundary correction mechanism that handles calculation errors caused by angle values jumping between 180 degrees and -180 degrees, correctly accumulating the actual rotation angle. Once the rotation reaches the preset value, all motors automatically stop, making the robot's turning action more stable and accurate.

c. Autonomous Rescue Core Module

This program is the core functional module for the robot's autonomous rescue

```
def turn_left_find():
    global current_direction
    for i in range(2):
        turn_left(32)
        find_victim()
    adjust_left_right()
    current_direction = (current_direction - 1 + 4) % 4
```

(Code excerpt)

```
def turn_left(angle):
    start_angle = get_angle(REQ_ANGLE_2)
    total_turned = 0
    last_angle = start_angle
    # 启动角速
    motor_control(-sp1, -sp2, sp3, sp4)
    while total_turned < angle:
        current_angle = get_angle(REQ_ANGLE_2)
        # 计算这一次旋转了多少度 (范围 180/-180 弧度)
        diff = current_angle - last_angle
        # 如果发生跳变 (例如从 179 变到 -179) diff 会是 -358
        # 我们需要将它修正回正确的左转弯角度 (2 度)
        if diff < -180:
            diff += 360
        elif diff > 180:
            diff -= 360
        # 左转弯: 角度是持续增加的
        # 如果从测距数据左转弯增加, 则 diff 会正
        total_turned += diff
        last_angle = current_angle
        # 为了避免死循环, 可以加入一个最小的延迟
        time.sleep_ms(1)
    # 停止角速
    motor_control(0, 0, 0, 0)
```

mission. It integrates key mechanisms such as ground color recognition, victim detection, visual feedback prompts, and automatic rescue-kit deployment. It is responsible for judging environmental conditions, identifying target categories, and executing corresponding mission actions, making it an important program block for completing field tasks.

The blue() function is specifically used to detect whether the ground is blue. It calls the color sensor to read real-time RGB values and illumination information. When the detected blue value exceeds the set threshold, it automatically executes a delay procedure for competition or scenario-specific trigger conditions, such as detecting a blue area, finish zone, or special floor marker, thereby improving the robot's environmental judgement capability.

The led_blink() function is responsible for visual warnings and status feedback. It controls the LED indicator to blink five times continuously, with each on and off state lasting 0.4 seconds, creating a regular prompt. When the robot successfully detects a victim, this function is triggered immediately so that observers can clearly recognize that the robot has entered rescue mode. It also serves as internal confirmation feedback for system actions.

```

def blue():
    r, g, b, lux = get_color_and_lux()
    if b > 110:
        time.sleep(4)

def led_blink():
    for i in range(5):
        led.value(1)
        time.sleep(0.4)
        led.value(0)
        time.sleep(0.4)

```

The `drop_kits()` function precisely controls rescue-kit deployment. According to the input direction parameter and number of deployments, it drives the SG90 servo to rotate to the corresponding angle. When the direction parameter is 1, the servo rotates to 180 degrees to deploy; when the parameter is 2, it rotates to 0 degrees to deploy. After each action, it returns to the 90-degree standby position, ensuring stable and consistent deployment each time and allowing different numbers of deployments depending on task requirements.

```

def drop_kits(direc, times):
    if direc == 1:
        for i in range(times):
            angle = servo.set_angle(180)
            time.sleep(0.3)
            angle = servo.set_angle(90)
            time.sleep(0.3)
    elif direc == 2:
        for i in range(times):
            angle = servo.set_angle(0)
            time.sleep(0.3)
            angle = servo.set_angle(90)
            time.sleep(0.3)

```

The `find_victim()` function is the core judgement function of the entire rescue mission. It simultaneously reads camera recognition values and multiple distance-sensor data sets, and uses predefined conditional rules to identify different types of victims. The function determines the victim type based on the number returned by the camera, confirms the target position using distance conditions, and then performs the corresponding LED blinking and classified rescue-kit deployment. When a valid target is detected, the current coordinate is automatically recorded in the rescued list to prevent repeated rescue actions, ensuring that the robot can complete all rescue operations efficiently and accurately while moving.

```
def find_victim():
    global current_x, current_y
    read_data_5()
    print(cam_data())
    time.sleep(0.1)
    if (cam_data() == 14 or cam_data() == 11) and (d4 < 150 or d5 < 150):
        led_blink()
        drop_kits(1,2)
    elif(cam_data() == 15 or cam_data() == 13) and (d4 < 150 or d5 < 150):
        led_blink()
        drop_kits(1,1)
    elif (cam_data() == 16 or cam_data() == 12) and (d4 < 150 or d5 < 150):
        led_blink()
    elif( cam_data() == 4 or cam_data() == 1 )and (d6 < 150 or d7 < 150):
        led_blink()
        drop_kits(2,2)
    elif( cam_data() == 5 or cam_data() == 3 )and (d6 < 150 or d7 < 150):
        led_blink()
        drop_kits(2,1)
    elif( cam_data() == 6 or cam_data() == 2 )and (d6 < 150 or d7 < 150):
        led_blink()
    if cam_data() > 0:
        dropped_path.append([current_x, current_y])
```

d. Automatic Navigation and Path-Correction Core Module

This program is the core control module for automatic navigation and path correction. It is designed to keep the robot moving steadily in a maze or narrow passage, automatically correct deviations, and dynamically adjust its direction based on real-time laser distance sensor values, ensuring that the robot remains centered on the path and avoids hitting walls or deviating from the route.

The `adjust_left_right()` function performs real-time left-right fine adjustment. By reading multiple laser sensor data sets, it determines the robot's distance from the left and right walls, calculates the error, and dynamically adjusts motor

steering so that the robot automatically stays centered in the passage and does not move too close to either wall while going straight.

The `adjust_back()` function is used for rear distance detection and correction. It reads the rear laser sensor values, compares the distance difference between the two rear sides, and controls the motors to make slight turns based on the error size. This prevents deviation caused by uneven rear distance during movement and improves stability during reversing and turning.

The `adjust_front()` function handles front distance judgement and deceleration adjustment. It reads front laser values in real time. When an obstacle is detected within the safety-distance range, it automatically adjusts motor output so that the robot slows down smoothly and avoids directly hitting the obstacle, improving movement safety.

Overall, the module uses multi-directional distance detection from the front, rear, left, and right, together with real-time error correction, to enable stable, smooth, and autonomous navigation in complex environments. It greatly improves the accuracy, smoothness, and reliability of maze movement.

```
def adjust_left_right():
    read_data_5() #讀LASER 和GYRO值函數
    if d6 < 150 :
        if d7+8 > d6 :
            error = d7 + 8 - d6
            motor_mc(-sp1, -sp2, sp3, sp4,2,error*2) # 左TURN 45度
        elif d7+8 < d6:
            error = d6 - d7 - 8
            motor_mc(sp1, sp2, -sp3, -sp4,2,error*2) # 左TURN 45度
    elif d4 < 150 :
        if d4+ 40 > d5:
            error = d4+40 - d5
            motor_mc(-sp1, -sp2, sp3, sp4,2,error*2) # 左TURN 45度
        elif d4+40 < d5 :
            error = d5 - d4 - 40
            motor_mc(sp1, sp2, -sp3, -sp4,2,error*2) # 左TURN 45度

def adjust_back():
    read_data_5() #讀LASER 和GYRO值函數
    if d8 > d10 and d9 < 110:
        error = d8 - d10
        motor_mc(-sp1, -sp2, sp3, sp4,2,error*5) # 左TURN 45度
    elif d10 > d8 and d9 < 110:
        error = d10 - d8
        motor_mc(sp1, sp2, -sp3, -sp4,2,error*5) # 左TURN 45度

def adjust_front():
    read_data_5() #讀LASER 和GYRO值函數
    if d2 < 250 and d2 > 40:
        motor_mc(sp1, sp2, sp3, sp4,2,d2*4)
```

e. Autonomous Straight-Line Control and Navigation Core Function

This code is the core straight-line control function for autonomous movement, navigation, obstacle avoidance, and coordinate recording. It integrates speed control, real-time sensing, obstacle avoidance, orientation updates, and path recording so that the robot can move steadily in a maze, judge routes automatically, update coordinates, and perform victim detection and color recognition.

At the beginning of the `forward()` function, the base speed and direction of the four wheel motors are set to ensure that the robot moves forward smoothly. The function dynamically adjusts motor speed according to the laser distance values on the left and right sides to avoid moving too close to the walls. During movement, the program continuously reads color-sensor and illumination data. When low illumination and non-blue ground are detected, it automatically executes backing up, left turning, and obstacle-avoidance actions, and replans the forward route.

During movement, the function updates the X and Y coordinates in real time according to the current orientation (north, east, south, or west), and records visited positions in the path list to avoid repeated movement. At the same time, the program continuously calls victim detection, front and side correction, blue-floor detection, and other functions to ensure that the robot stays on a stable path, responds to environmental changes in real time, and completes mission detection while moving forward.

Overall, this function integrates closed-loop navigation, automatic obstacle avoidance, coordinate positioning, and task execution into a single autonomous movement logic. It is a key module for completing maze exploration and rescue missions.

```

def forward():
    global sp1, sp2, sp3, sp4, speed
    global current_x, current_y, current_direction
    speed = 90
    sp1 = speed
    sp2 = -speed
    sp3 = -1*speed
    sp4 = 1*speed
    read_data_5()
    if (d4 > 20 and d4 < 100) or (d6 > 100 and d6 < 300):
        sp3 = 0.9*sp3
        sp4 = 0.9*sp4
    if (d6 > 20 and d6 < 100) or (d4 > 100 and d4 < 300):
        sp1 = 0.9*sp1
        sp2 = 0.9*sp2
    motor_mc(sp1, sp2, sp3, sp4, 2, 1200)
    find_victim()
    r, g, b, lux = get_color_and_lux()
    if lux < 4 and b < 110:
        motor_mc(-sp1, -sp2, -sp3, -sp4, 2, 1100)
        time.sleep(0.2)
        turn_left(70) # 左TURN 90度
        current_direction = (current_direction - 1 + 4) % 4
        time.sleep(0.2)
        read_data_5()
        if d2 > 250:
            motor_mc(sp1, sp2, sp3, sp4, 2, 2700)
            # 4. 更新坐标
            if current_direction == 0: # 北
                current_y += 1
            elif current_direction == 1: # 东
                current_x += 1
            elif current_direction == 2: # 南
                current_y -= 1
            elif current_direction == 3: # 西
                current_x -= 1
            print(current_x)
            print(current_y)
            print(current_direction)
            # 记录路径
            visited_path.append([current_x, current_y])
        else:
            turn_left(70) # 右TURN 90度
            current_direction = (current_direction - 1 + 4) % 4

```

```

    else:
        motor_mc(sp1, sp2, sp3, sp4, 2, 1100)
        # 4. 更新坐标
        if current_direction == 0: # 北
            current_y += 1
        elif current_direction == 1: # 东
            current_x += 1
        elif current_direction == 2: # 南
            current_y -= 1
        elif current_direction == 3: # 西
            current_x -= 1
        print("dir:{current_direction},x:{current_x},y:{current_y},判断是否(is_visited)
        #print(current_x)
        #print(current_y)
        #print(is_visited(current_x,current_y))
        #print(is_dropped(current_x,current_y))
        print(visited_path)
        print(dropped_path)
        time.sleep(1)
        # 记录路径
        visited_path.append([current_x, current_y])

    adjust_front()
    adjust_left_right()
    find_victim()
    blue()

```

f. Top-Level Maze Exploration Decision Function

This program is the top-level core decision function for robot maze exploration. It is responsible for overall path planning, environmental judgement, and movement command assignment, and is the key module for fully automatic maze exploration, route traversal, and autonomous navigation.

The `maze_explore()` function first determines whether the robot has returned to the starting point. If it has returned to the origin, it performs a long pause to indicate that the mission is complete. The program then reads the laser sensor values, detects the obstacle status in the front, left, and right directions in real time, filters out all passable directions, and builds a list of available paths.

For each available path, the function simulates the robot's coordinates and orientation after turning left, turning right, or moving straight, and checks whether the position has already been visited. Through coordinate comparison, it builds complete path information. In the decision stage, the program uses the core rule of prioritizing unvisited paths and strictly follows the order “left turn first -> straight -> right turn” to choose the most suitable

movement direction, ensuring that the robot can effectively explore all maze areas without repeatedly circling.

After the final movement direction is determined, the function automatically calls the corresponding turning and straight-line functions to perform turning search, forward movement, coordinate update, and path recording. If the current position is determined to be a dead end with no available direction, the program automatically triggers a 180-degree turn, replans the route, and continues exploration.

Overall, this function integrates five major functions: sensor-data reading, obstacle judgement, path simulation, intelligent decision making, and action execution. It builds a complete and stable closed-loop maze exploration system, allowing the robot to move autonomously in unknown environments, avoid obstacles, record paths, and complete global exploration efficiently.

```

# 根據當前方向和可能的移動，計算下一個座標並檢查訪問狀態
paths_info = []
for move in possible_moves:
    temp_x, temp_y, temp_direction = current_x, current_y, current_direction

    if move == "front":
        if temp_direction == 0: # 北
            temp_y += 1
        elif temp_direction == 1: # 東
            temp_x += 1
        elif temp_direction == 2: # 南
            temp_y -= 1
        elif temp_direction == 3: # 西
            temp_x -= 1
    elif move == "right":
        # 根據右轉後的方向
        new_dir = (temp_direction + 1) % 4
        if new_dir == 0: # 北
            temp_y += 1
        elif new_dir == 1: # 東
            temp_x += 1
        elif new_dir == 2: # 南
            temp_y -= 1
        elif new_dir == 3: # 西
            temp_x -= 1
    elif move == "left":
        # 根據左轉後的方向
        new_dir = (temp_direction - 1 + 4) % 4
        if new_dir == 0: # 北
            temp_y += 1
        elif new_dir == 1: # 東
            temp_x += 1
        elif new_dir == 2: # 南
            temp_y -= 1
        elif new_dir == 3: # 西
            temp_x -= 1

    paths_info.append({"move": move, "coords": [temp_x, temp_y], "visited": is_visited(temp_x, temp_y)})

```

```

# 決策邏輯：優先選擇未走過的路，順序為：左 -> 前 -> 右
if unvisited_paths:
    # 1. 優先左方未走過
    for p in unvisited_paths:
        if p["move"] == "left":
            chosen_move = "left"
            break
    # 2. 前方未走過
    if not chosen_move:
        for p in unvisited_paths:
            if p["move"] == "front":
                chosen_move = "front"
                break
    # 3. 右方未走過
    if not chosen_move:
        for p in unvisited_paths:
            if p["move"] == "right":
                chosen_move = "right"
                break
# 如果都走過了，則從已走過的路徑中選擇（優先順序：左 -> 前 -> 右）
elif paths_info:
    for p in paths_info:
        if p["move"] == "left":
            chosen_move = "left"
            break
    if not chosen_move:
        for p in paths_info:
            if p["move"] == "front":
                chosen_move = "front"
                break
    if not chosen_move:
        for p in paths_info:
            if p["move"] == "right":
                chosen_move = "right"
                break

```

● April 6, 2026

Full debugging began: After building the new robot hardware and program, we began complete debugging. During this process, we tested and fine-tuned parameters for motor driving, various sensors, servo actions, turning accuracy, and overall maze exploration logic. We corrected program execution errors, sensor threshold deviations, and module-coordination issues to ensure stable movement, accurate path judgement, and smooth, normal

operation of all mission functions.



We evaluated the robot's agility, speed, and accuracy in different scenarios and compared its overall performance closely with competition challenges.

For agility testing, we designed several maze environments, including straight corridors, right-angle turns, consecutive curves, slopes, and speed bumps. The results showed that the first-generation robot performed acceptably in straight corridors, but its reaction speed was slightly insufficient in consecutive curves and on slopes. It occasionally turned slowly or oversteered. After structural and program optimization, the second-generation robot reduced turning response time by about 30%, and its slope movement became more stable and smooth.

For speed testing, we recorded the total time required for the robot to complete a fixed-length maze path, and

compared driving stability and task completion rate under different speed

settings. The results showed that the first-generation robot tended to slip or deviate from the route at higher speed settings, increasing the mission failure rate. After the second-generation robot switched to GA25-370 motors and optimized the wheel structure, its average driving speed increased by about 20% while maintaining stability, and the route-deviation range during high-speed driving was clearly reduced.

For accuracy testing, we focused on evaluating obstacle-avoidance navigation, path-judgement accuracy, victim-recognition success rate, and rescue-kit deployment precision. The results showed that the first-generation robot sometimes misjudged obstacle detection, occasionally treating open areas as obstacles and taking unnecessary detours. Its victim-recognition success rate was about 80%. After the second-generation robot upgraded to laser sensors, the obstacle-detection false-judgement rate was greatly reduced, the victim-recognition success rate increased to over 90%, and rescue-kit deployment precision improved from the original 75% to nearly 90%.

We used statistical methods to analyze testing results and identify performance patterns. For example, we found that the robot's success rate for right turns was generally lower than for left turns. After inspection, we confirmed that this was caused by uneven tire wear, which led us to improve the wheel structure. These analyses provided concrete evidence for design modifications and software adjustments, further improving overall efficiency.

Although the evaluation produced good results and the second-generation robot clearly outperformed the first generation in many performance indicators, the overall progress was slightly slower than originally planned. The main reason was that problems discovered during testing required repeated structural and program modifications, which took considerable time. We will take measures to speed up later evaluation and improvement, including optimizing the testing process, preparing multiple sets of spare parts in advance, and assigning different tests to team members simultaneously, ensuring that all adjustments and verifications are completed before the

competition.

We use the ESP-32 control board as the main control module and independently designed and produced two dedicated PCB boards, each responsible for different circuit functions, integrating power supply, signal transmission, and module connections.

The robot vehicle uses an array of 9 infrared reflective sensors to identify the route, one vision module on each side to identify victims, a color sensor on the underside to identify floor colors according to the competition rules, two front bumpers or touch limit switches to prevent collision with obstacles, and DC geared motors for driving.

To ensure that the robot vehicle can cross obstacles smoothly, we designed the overall body clearance to be 2.5 cm from the ground. At the same time, we added parallel rods to the two wheels. This mechanism ensures that all four wheels remain in contact with the ground and prevents wheels from lifting off when crossing obstacles.

We also limited the overall height of the body and lowered the center of gravity as much as possible, for example by placing the battery box in the center area underneath the vehicle. This layout optimization not only effectively reduces the overall center of gravity, but also balances the front-rear and left-right weight distribution, keeping the robot's center of gravity stable at the center of the body. Even in extreme scenarios such as climbing or obstacle crossing, excessive center-of-gravity shift can be avoided.

We used 3D drawing software to create wheel molds, then manufactured the robot vehicle wheels using the chemical properties of resin and hardener. This increased wheel friction, and we designed recessed areas in the wheels so that the vehicle can cross obstacles more easily. At the same time, we added an annular groove structure to the wheel body that fully matches the silicone

tire. With this design, the silicone tire can be completely embedded in the groove, creating tight engagement between the tire and hub and fundamentally limiting lateral movement and detachment risk. The groove also improves the tightness of the tire-hub connection, strengthens the overall wheel structure, and helps the robot maintain stable grip and driving performance in complex scenarios such as turning and climbing, while extending tire life and reducing maintenance frequency and difficulty.

- April 25, 2026

The team went to Hong Kong to participate in the preliminary competition. The robot successfully completed the required tasks and qualified for the China competition. During the competition run tests, we also observed two clear shortcomings: first, each movement and action step took too long, making the overall operating rhythm slow; second, the circuit-board wiring connections



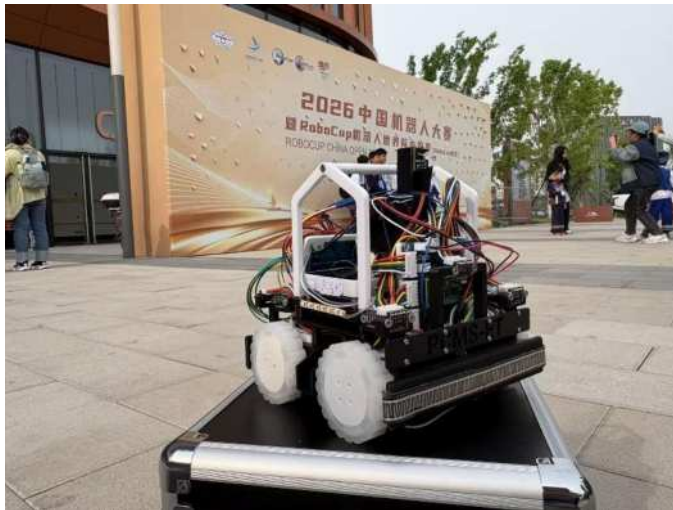
were not stable enough and were prone to poor contact. These issues became the main focus of post-competition optimization and adjustment.

- April 29, 2026

We carried out corresponding optimization adjustments for the two problems found in the competition. To address the excessive execution time of robot steps, we reorganized the program execution logic, removed unnecessary delay commands, simplified repeated detection processes, shortened the execution time of each action module, and accelerated the overall operating rhythm. To address poor contact in circuit-board wiring, we reorganized the internal wiring, replaced loose wires with stable connectors, strengthened solder joints, and planned a cleaner wiring layout to reduce wire detachment and contact interruption caused by vibration, improving overall circuit stability.

- May 1 to May 5, 2026

The team went to Beijing to participate in the national China competition. The robot successfully completed all competition tasks and qualified for the World Championship. Although the competition was completed successfully, it revealed a clear weakness: the robot's overall driving stability was insufficient, and it tended to shake and deviate while moving. In addition, after carefully studying the World Championship rules, we learned that the international competition field would be much more difficult, with many complex obstacles and slope sections. The existing robot's structure, power, and weight-distribution design could not handle such a challenging field. To meet the new challenge and improve competitiveness, the team decided after discussion to abandon the old architecture and redesign and develop a new generation of dedicated robot.



- May 14, 2026

The team officially completed the redesign and assembly of the new-generation robot. To address the old robot's unstable driving and to meet the high-difficulty World Championship field requirements with many obstacles and slopes, we comprehensively optimized the body structure, weight distribution, and chassis drive mechanism. These changes improved overall

driving stability, climbing ability, and obstacle-crossing performance, enabling the new robot to better adapt to the World Championship environment.

Because the Incheon World Championship will include more obstacles and higher difficulty, we decided to redesign the body and add a rocker-style parallel linkage mechanism.



Multi-Sensor Integration and Drive Configuration (Sensors & Actuators)

The vehicle is configured with the following sensing and drive components:

- **Path recognition:** The front is equipped with an array of 9 infrared reflective sensors (V2 Laser Ranging Sensor VL53LoX) to accurately track the path.
- **Victim recognition:** One vision-recognition module(Sipeed MaixCAM), is installed on each side of the body to identify victims and visual signs on the walls.
- **Terrain perception:** A color sensor is installed on the underside of the vehicle to identify black no-go areas (drop tiles) or special floor tiles. Touch limit switches (bumpers) are installed on both front sides to prevent hard collisions with maze walls.
- **Power drive:** The vehicle is driven by high-torque GA25 DC geared motors with encoders. Encoder feedback enables precise odometry positioning.

Core Control and Circuit System (Control & Circuit System)

The main control module uses an ESP32 microcontroller with high-speed dual-core processing and wireless communication capability. To improve system stability, we independently designed and manufactured two dedicated PCB boards. This design effectively integrates power management, signal transmission, and connections to each sensor module, greatly reducing the clutter caused by traditional Dupont wiring and improving the robot's anti-interference capability during long competition runs.

Chassis Suspension and Dynamic Balance (Chassis & Suspension)

To pass obstacles and speed bumps in the maze smoothly, we optimized the chassis ground clearance to 2.5 cm. The chassis uses a rocker-bogie / parallel linkage mechanism. This suspension design ensures that all four wheels stay in close contact with the ground when crossing uneven terrain. In addition, we fixed the heavy battery box at the center of the underside of the vehicle, maximizing the reduction of the overall center of gravity and achieving balanced front-rear and left-right weight distribution, ensuring that the robot remains highly stable when climbing or crossing obstacles.

Rescue Kit Deployment System: To handle victims that may randomly appear on either side of the maze, we designed a bidirectional diverter rail and used a central turntable mechanism to precisely control the drop direction of the blocks, enabling stable deployment on both the left and right sides. In addition, guide rails are integrated into the underside of the vehicle to ensure that the blocks fall smoothly and accurately into the designated victim area.

Wheel Manufacturing Process and Structural Design (Wheels & Tire Design)

We modeled the wheels with 3D drawing software and used epoxy resin molding to produce high-strength special wheel hubs. The hub profile includes toothed grooves for obstacle crossing, helping the vehicle grip and pass over obstacles. At the same time, the hub surface includes an annular retaining groove that perfectly matches the silicone tires. This design locks the silicone tires firmly inside the hub, preventing lateral movement or detachment during turning or climbing and ensuring stable traction.